Experiment-8

Keyword Based Monoalphabetic Substitution Cipher

Aim:- To implement a monoalphabetic substitution cipher using a keyword.

Algorithm:-

1. Read the plaintext and keyword.
2. Remove repeated letters from the keyword.
3. Append the remaining unused alphabet letters in order.
4. Create a cipher alphabet using the keyword sequence.
5. Match each plaintext letter with its corresponding cipher letter.
6. Replace each letter using the substitution mapping.
7. Display the encrypted ciphertext.

Pseudocode:-

```
Scanf ("%s", text);
scanf ("%s", key);
create_cipher_alphabet(key);
for (i=0; text[i] != '\0'; i++)
    cipher[i] = substitution(text[i]);
printf ("%s", cipher);
```

Input:-

Plaintext :- ABC
Keyword :- CIPHER

Output:-

Ciphertext :- CIP

Experiment-9

Playfair Cipher Decryption

Aim:- To implement playfair cipher decryption for a given ciphertext.

Algorithm:-

1. Read the given ciphertext and keyword.
2. Construct a 5 x 5 matrix using the keyword.
3. Divide the ciphertext into pairs of letter.
4. If both letters are in the same row shift each letter one position to the left.
5. If both letters are in the same column, shift each letter one position upward.
6. Otherwise, use the rectangle rule to find the corresponding letters.
7. Combine the decrypted pairs and display the plaintext.

Pseudocode:-

```
Scanf("%s", Cipher);
Create_matrix(key);
for(i=0; cipher[i]!='\0'; i+=2)
    decrypt_pair(cipher[i], cipher[i+1]);
printf("%s", plaintext);
```

Input:-

Ciphertext:- KXJEYUREBEZWEHEWRY
KEY:- ROYAL

Output:-

Decrypted plaintext:- PT BOAT ONE

Experiment - 10
playfair cipher encryption

Aim:- To encrypt a message using the given playfair matrix.

Algorithm:-

1. Read the plaintext.
2. Use the given 5 x 5 matrix
3. Divide Plaintext into letter pairs.
4. Apply the playfair encryption rules to each pair.
5. Replace each pair with encrypted letters.
6. Combine the encrypted pairs.
7. Display the ciphertext

Pseudocode:-

```
scanf("%s", text);
for(i=0; text[i] != '\0'; i+=2)
    encryptpair(text[i], text[i+1]);
cipher[i] = '\0';
printf("%s", cipher);
```

Input:-

Plaintext = Must see you over cadogan west coming at once.

Output:-

UZTBDLSZPNWLSJTSTUEPOVLDBDUHF
PERHWQSRZ.

Experiment-11

Playfair cipher key Analysis

Aim:- To calculate the possible and effectively Unique keys in the playfair cipher.

Algorithm:-

1. Consider the 25 letters used in the playfair matrix.
2. Calculate the total number of possible letter arrangements.
3. Express the number of possible keys as a power of 2.
4. Consider that different keys may produce identical encryption results.
5. Calculate the effectively unique keys.
6. Compare the total and unique keys
7. Display the results.

Pseudocode:-

```
keys = factorial(25);
Unique = keys/equivalent_keys;
printf("Total keys = %Lf", keys);
printf("Unique keys = %Lf", Unique);
```

Input:-

Letters = 25

Output:-

Total possible keys = 25!

Experiment - 12

Hill cipher Encryption and Decryption

Aim:- To implement Hill cipher encryption and Decryption using a key matrix.

Algorithm:-

1. Read the plaintext and key matrix
2. Convert each letter into a numerical value
3. Divide the plaintext into blocks matching the matrix size.
4. Multiply each block by the key matrix.
5. Apply modulo 26 to obtain the cipher text values.
6. For decryption, calculate the inverse of the key matrix modulo 26.
7. Convert the resulting values into letters and display the plaintext.

Pseudocode:-

```
scanf("%s", text);
convert(text);
for (i=0; i<strlen(text); i+=2)
    encrypts_block(text[i], text[i+1]);
printf("%s", cipher);
```

Input:-

plaintext :- MEET

key :- 9 4
5 7

Output:-

Cipher text :- UKIX

Experiment-13
Hill cipher attack

Aim:- To demonstrate a known-plaintext and chosen-plaintext attack on the Hill cipher.

Algorithm:-

1. Read the plaintext and corresponding ciphertexts.
2. Convert letters into numerical values
3. Arrange the values into matrices.
4. Select sufficient plaintext-ciphertext pairs.
5. Calculate the inverse of the plaintext matrix mod 26
6. Determine the encryption key matrix.
7. Display the recovered key.

Pseudocode:-

```
Scanf("%s %s", plain, cipher);
for (i=0; plain[i] != '\0'; i++)
    convert (plain[i], cipher[i]);
Find matrix ();
printf("Recovered key matrix");
```

Input:-

Plaintext :- HELP
Ciphertext:- HIAP

Output:-

6 2
13 16

Experiment-14

One-time pad Vigenere Cipher

Aim:- To encrypt and decrypts a message using the one-time pad version of the Vigenere cipher.

Algorithm:-

1. Read the plaintext and key stream.
2. Convert each letter into a number from 0 to 25
3. Add the corresponding key value to each plaintext value.
4. Apply modulo 26 to obtain the ciphertext value.
5. Convert the values back into letters.
6. For decryption, substract the key values and Apply modulo 26.
7. Display the ciphertext and decrypted plaintext

Pseudocode:-

```
scanf("%s", text);
for(i=0; text[i] != '\0'; i++)
    cipher[i] = (text[i] - 'A' + key[i]) % 26 + 'A';
Cipher[i] = '\0';
printf("%s", cipher);
```

Input:-

plaintext : SEND
key: 9017

Output:-

Ciphertext :- BFOU

classmate
Date
Page

Experiment-15

Letter Frequency Attack on an Additive cipher

Aim:- To write a C program that performs a letter frequency attack on an additive cipher and displays possible plaintexts in order of Likelihood.

Algorithm:-

1. Read the encrypted ciphertext.
2. Count the frequency of each letter in the ciphertext
3. Try all 26 possible Caesar cipher shifts.
4. Decrypts the ciphertext using each shift value.
5. Compare the resulting text with common English letter frequencies.
6. Rank the possible plaintexts from most likely to least likely.
7. Display the top possible plaintexts.

Pseudocode:-

```
scanf("%s", cipher);
for(k=0; k<26; k++){
    decrypt(cipher, k, plain);
    score[k] = frequency_score(plain);
}
display_top_plaintexts(score);
```

Input:-

KHOOR

Plain text:-

Most likely plaintext:- HELLO.